Alberto Andrés Valdés González.
Degree: Mathematical Engineer.
Work position: ML-Engineer.
Mail: anvaldes@uc.cl/alberto.valdes.gonzalez.96@gmail.com
Location: Santiago, Chile.

Terraform and its three classic files

Setting: Terraform (HashiCorp; open-source fork: OpenTofu) is an **Infrastructure as Code** tool. Instead of creating cloud resources by clicking in a console, you **declare** the desired infrastructure in `.tf` files written in HCL; Terraform compares it with what exists and creates, changes or destroys resources to match. The code is versioned in Git, reviewed in PRs and reproducible across environments (dev, staging, prod). **Providers** (AWS, GCP, Azure, Kubernetes, Snowflake, ...) translate the code into API calls.

Terraform loads *all* `.tf` files in a folder together, so the split is a convention, but the classic one is:

$$\underbrace{\texttt{variables.tf}}_{\text{inputs}} \longrightarrow \underbrace{\texttt{main.tf}}_{\text{resources}} \longrightarrow \underbrace{\texttt{outputs.tf}}_{\text{results}}$$

Like a function: `variables.tf` are the parameters, `main.tf` is the body and `outputs.tf` is the return value.

1. `main.tf`: what to create

Declares the **provider** and the **resources** (and optionally **data sources**, to read existing resources, and **modules**, reusable groups of resources). Each resource has a type and a local name: `resource "<type>" "<name>"`.

```
terraform {
  required_providers { aws = { source = "hashicorp/aws", version = "~> 5.0" } }
  backend "s3" { bucket = "my-tf-state", key = "ml/terraform.tfstate", region = "us-east-1" }
}

provider "aws" { region = var.region }

resource "aws_s3_bucket" "datasets" {
  bucket = "${var.project}-${var.environment}-datasets"
  tags   = { project = var.project, env = var.environment }
}

resource "aws_instance" "trainer" {
  ami           = var.ami_id
  instance_type = var.instance_type
  tags          = { Name = "${var.project}-trainer" }
}
```

2. `variables.tf`: inputs

Declares the parameters, with type, description, optional default and validation. They are referenced as `var.<name>`. Nothing is hard-coded, so the same code serves every environment.

```
variable "project"     { type = string }
variable "region"      { type = string, default = "us-east-1" }
```

```
variable "ami_id"      { type = string }
variable "environment" {
  type        = string
  description = "Deployment environment"
  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "environment must be dev, staging or prod."
  }
}
variable "instance_type" {
  type    = string
  default = "t3.medium"
}
```

3. `outputs.tf`: results

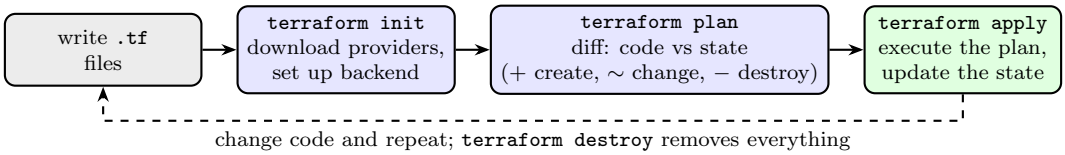
Exposes values after `apply` (IDs, ARNs, URLs, IPs): printed in the terminal, read by other modules, or passed to CI/CD. `sensitive = true` hides secrets in logs.

```
output "bucket_name" { value = aws_s3_bucket.datasets.bucket }
output "bucket_arn"  { value = aws_s3_bucket.datasets.arn }
output "trainer_ip"  { value = aws_instance.trainer.public_ip }
```

Related files that complete the picture:

- `terraform.tfvars` (or `dev.tfvars`, `prod.tfvars`): the *values* of the variables, e.g. `project = "fraud"`, `environment = "dev"`. `variables.tf` declares, `.tfvars` assigns. Secrets should not go here in Git.
- `providers.tf / versions.tf`: in larger projects, the `terraform` and `provider` blocks are moved out of `main.tf`.
- `terraform.tfstate`: generated by Terraform, never edited by hand. It maps each resource in the code to the real resource in the cloud (its ID). It should live in a **remote backend** (S3, GCS, Terraform Cloud) with **locking**, so two people cannot apply at the same time, and never be committed to Git (it can contain secrets).
- `.terraform.lock.hcl`: pins exact provider versions; it *is* committed.

Workflow:



Other useful commands: `terraform fmt` (format), `terraform validate` (syntax check), `terraform apply -var-file=prod.tfvars` (choose environment). In CI/CD, `plan` runs on every PR and `apply` after merge.

The three classic files: `main.tf` declares *what* infrastructure exists (provider + resources), `variables.tf` declares the *inputs* that parametrize it, and `outputs.tf` exposes the *results*. Values go in `.tfvars`, and Terraform tracks reality in the `tfstate`, which must live in a remote, locked backend.